

Chess Rating Estimation from Moves and Clock Times Using a CNN-LSTM

Michael Omori^[0009–0000–4632–9272], Prasad Tadepalli^[0000–0003–2736–3912]

Oregon State University, Corvallis OR, USA
 {omorim, prasad.tadepalli}@oregonstate.edu

Abstract. Current chess rating systems update ratings incrementally and may not always accurately reflect a player’s true strength at all times, especially for rapidly improving players or very rusty players. To overcome this, we explore a method to estimate player ratings directly from game moves and clock times. We compiled a benchmark dataset from Lichess with over one million games, encompassing various time controls and including move sequences and clock times. Our model architecture comprises a CNN to learn positional features, which are then integrated with clock-time data into a Bidirectional LSTM, predicting player ratings after each move. The model achieved an MAE of 182 rating points on the test data. Additionally, we applied our model to the 2024 IEEE Big Data Cup Chess Puzzle Difficulty Competition dataset, predicted puzzle ratings and achieved competitive results. This model is the first to use no hand-crafted features to estimate chess ratings and also the first to output a rating prediction after each move. Our method highlights the potential of using move-based rating estimation for enhancing rating systems and potentially other applications such as cheating detection.

Keywords: Chess · Rating Estimation · Cheating Detection.

1 Introduction

Traditional rating systems like Elo [6] and Glicko [7], though effective, often require many games to accurately reflect the true level of skill of a player, leading to potential mismatches and inaccuracies, especially for new accounts or rapidly improving players. This delay in accurate rating adjustment can skew competitive fairness and hinder effective matchmaking. For example, lower-rated players were shown to have longer winning streaks than higher-rated players on average, suggesting that their rating may be less accurate compared to higher-rated players [3]. This could occur if a person creates a new account and is either an experienced chess player who made a second private account or a new player who transferred skills from other board games. In both cases, their initial rating starts lower on sites such as lichess [16] and chess.com [2], but their performance could be much higher.

To overcome these limitations, we propose a novel method that estimates player ratings directly from game moves and clock times, aiming to provide accurate ratings after each game. Our dataset comes from Lichess, encompasses 1.2 million games from April 2021 to July 2024 (inclusive), covers various time controls and includes detailed move sequences and clock times. Chess streamers like GothamChess [23] have attempted this on their own (without computer aid) by using their chess knowledge to look at games and "guess the elo" and it is quite challenging to be consistently accurate.

Our method is a neural network architecture that combines a four-layer Convolutional Neural Network (CNN) and a Bidirectional Long Short-Term Memory (LSTM) network. The CNN extracts positional features from the chessboard, which are then integrated with clock time data by the LSTM to predict player ratings move-by-move. This approach allows the model to consider both spatial and temporal dimensions of the game, leading to more precise rating predictions. The model achieved a mean absolute error (MAE) of 183 rating points in the test data, demonstrating the effectiveness of the overall approach.

Ablation studies confirm that the inclusion of clock time data improves the model performance. Additionally, the same model is tested on the 2024 IEEE Big Data Cup Chess Puzzle Difficulty Competition dataset, where it predicts puzzle ratings competitively, indicating the model's versatility and robustness.

An additional application of the rating estimate for each move is to check for anomalous behavior. For example, a 1300 rated player that is playing like a 2600 in half their games, but 1300 in the other half may be considered an anomaly. A CNN was [20] trained on Lichess games but only achieved marginally better results than random guessing for classifying cheaters. Lichess itself has software called Kaladin for detecting cheaters [14]. It uses a CNN and also utilizes chess insights, which includes metrics such as average centipawn loss and number of moves. One downside to this approach is that it can only do as well as the people who labeled the data.

Our main contribution is creating the first neural network that predicts chess rating move by move without hand-crafted features. This is unlike the rating players normally get after playing an entire game, which is based off their current rating, the game result, and the opponent's rating. Second, we collated a benchmark from Lichess data for evaluation and finally demonstrated the utility of the amount of clock time remaining after each move in estimating player rating.

In conclusion, this method presents an advancement in chess rating estimation by leveraging deep learning techniques to analyze in-game moves and clock times. The promising results and potential applications highlight the importance of this method, paving the way for more dynamic and fair rating systems in competitive chess.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

2 Background

There are various rating systems to estimate a player's skill, such as FIDE's Elo and Lichess's Glicko 2. Elo uses expected scores and a K-factor for rating updates, while Glicko incorporates rating deviation (RD) and volatility for more accurate adjustments. We will briefly explain Glicko2 since that is the rating system that we are trying to estimate through moves.

2.1 Glicko-2 Rating System

Glicko-2 incorporates volatility to indicate how consistent a player plays, along with a rating and a rating deviation for each player. A collection of games like in a tournament is used to update these values and is treated to have occurred simultaneously with the same ratings μ , volatility, and deviation ϕ for each game. The rating deviation is manually set for each player to a specified amount such as between 0.3 and 1.2.

First, the variance v is calculated based on the outcomes of the game for each opponent with the rating μ_j .

$$v = \left[\sum_{j=1}^m g(\phi_j)^2 E(\mu, \mu_j, \phi_j) \{1 - E(\mu, \mu_j, \phi_j)\} \right]^{-1} \quad (1)$$

where

$$g(\phi) = \frac{1}{\sqrt{1 + 3\phi^2/\pi^2}} \quad (2)$$

$$E(\mu, \mu_j, \phi_j) = \frac{1}{1 + \exp(-g(\phi_j)(\mu - \mu_j))} \quad (3)$$

The change in rating Δ is then calculated by comparing the actual outcome of the game (win 1, draw 0.5, loss 0) with the expected result based on the logistic formula in equation 3. Note that $g(\phi_j)$ and v are directly proportional to the step size of the rating change.

$$\Delta = v \sum_{j=1}^m g(\phi_j) (s_j - E(\mu, \mu_j, \phi_j)) \quad (4)$$

The new volatility is calculated using the Illinois algorithm [27], based on Δ , ϕ , and v . A more detailed explanation can be found in the paper by Mark Glickman [7]. The final step involves updating both the rating deviation and the player ratings to μ' and ϕ' .

$$\phi^* = \sqrt{\phi^2 + \sigma'^2} \quad (5)$$

$$\phi' = \frac{1}{\sqrt{\frac{1}{\phi^{*2}} + \frac{1}{v}}} \quad (6)$$